

Programare Orientată Pe Obiecte

Curs 9

Colecții în Java

Enumerări și iteratori

Folosirea Genericelor

Noiembrie 2025

Din feedback

TODO:

- Exerciții și fișe de lucru
- Quizz-uri
- Sa lucrăm mai mult pe subiecte de examen, modele.
- Să reia concepte și exemple de data trecută (atunci parcă le înțeleg mai bine).

Altele ?

Limitările tablourilor

- Tablourile nu sunt întotdeauna cea mai bună soluție pentru gestiunea seriilor, seturilor și a grupurilor de date.
- Exemple:

```
1 int arr[] = new int[10];
2 /* Tablou de intregi */
3 int[] arr = new int[3];
4 arr[0] = 1;
5 arr[1] = 2;
6 arr[2] = 3;
7 // Nu se mai pot adauga elemente
```

- Tablourile nu excelează atunci când mărimea setului de date fluctuează.
- Inserarea unui element necesită glisarea elementelor de deasupra punctului de inserție.

Limitările tablourilor (continuare)

- Tablourile expun problemele gestiunii de nivel jos a memoriei.
- Dacă se oferă acces la un tablou privat:
 - Poate fi accesibil printr-o metodă accesoare.
 - Poate furniza o interfață pentru iterare (metodele: `first`, `next` etc.).
 - Poate întoarce o copie adâncă a tabloului, dar este foarte ineficient.

Colecții versus tablouri

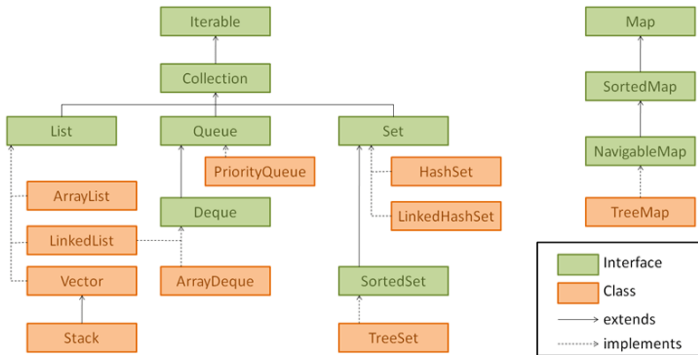
- Colecțiile (`API Collection`) sunt diferite de tablouri.
- Caracteristici ale tablourilor:
 - Legare tare la tipuri (specificare clară la compilare).
 - Se pot defini tablouri de elemente primitive.
- Caracteristici ale colecțiilor:
 - Legare slabă la tipuri.
 - Nu se pot include valori primitive direct, dar se folosesc clase învelitoare (`Integer`, `Double` etc.).
- Tablourile dau în general viteză de execuție mai mare, deoarece reprezintă blocuri de memorie accesibile direct
- Colecțiile sunt obiecte cu metode care trebuie invocate pentru a citi sau scrie elemente
- Colecțiile sunt mult mai ușor de folosit pentru programare de uz general, în special atunci când datele sunt foarte volatile
 - Multe adăugări, ștergeri și modificări directe în timp

Colecții

- O colecție Java este orice clasă care păstrează obiecte și implementează interfața `Collection`.
- Exemple:
 - Clasa `ArrayList<T>` implementează toate metodele din interfața `Collection`.
- Interfața `Collection` este cel mai înalt nivel din cadrul general Java pentru clase colecție.
- Toate clasele colecție tratate se află în pachetul `java.util`.
- API-ul `Collection` include:
 - Colecții cum sunt `Vector`, `LinkedList`, `Stack`.
 - Mapări: `HashMap`, care indexează valori pe baza cheilor.
 - Variante sortate: `TreeSet`, `TreeMap`.
 - Iteratori pentru citire și scriere.

Peisajul Collection

- Colecțiile **ordonate** implementează `List`.
- Colecțiile care asigură **unicitatea** elementului implementează `Set`.
- Colecțiile **sortate** implementează `SortedSet`.
- Colecțiile care **indexează valori pe baza cheilor** implementează `Map`.



Cadrul general Collection

- Interfața `Collection<T>` descrie operațiile de bază pe care toate clasele colecție trebuie să le implementeze.
- Orice metodă poate avea parametri de tip `Collection<T>`.
- Exemplu:

```
1 interface Collection<T> {  
2     public int size();  
3     public void clear();  
4     public Object[] toArray();  
5     public boolean add(Object o);  
6     public boolean remove(Object o);  
7     public boolean addAll(Collection c);  
8     public Iterator iterator();  
9 }
```

- Toate colecțiile pot:
 - adăuga / elimina elemente
 - șterge toate elementele colecției astfel încât rezultă un set vid
 - raporta mărimea lor
 - converti datele într-un tablou de `Object`

Construirea colecțiilor

- Colecțiile trebuie create explicit.
- Exemplu:

```
1 Vector<Integer> vec = new Vector<Integer>();  
2 vec.add(new Integer(5));  
3 vec.add(new Integer(4));  
4 System.out.println(vec); // Output: [5, 4]  
5 int i = ((Integer) vec.elementAt(0)).intValue();
```

- Elementele primitive trebuie învelite în clase (ex. Integer).
- Se pot elimina elemente cu metode definite de subtipuri ale Collection.

Colecții ordonate

Clase colecție concrete: Vector

- Caracteristici:
 - Se comportă ca un tablou cu acces aleatoriu.
 - Elementele nu trebuie să fie unice.
 - Inserările și ștergerile pot fi lente.
- Capacitatea crește automat atunci când este necesar.

```
1 import java.util.Vector;
2 public class ColecatiiDemo {
3     public static void main(String[] args) {
4         Vector<Integer> vec = new Vector<Integer>();
5         vec.add(10);
6         vec.add(2);
7         System.out.println("Vectorul contine");
8         for (Integer x : vec)
9             System.out.print(x + " ");
10    }
11 }
```

Clasa ArrayList

- Creare:

```
1 ArrayList<Integer> al = new ArrayList<Integer>();  
2 ArrayList<Integer> al2 = new ArrayList<>(int initial Capacity);
```

- Măsurarea: int size()

- Stocarea:

```
1 boolean add(Object o)  
2 void add(int index, Object element)  
3 Object set(int index, Object element)
```

- Regăsirea:

```
1 Object get(int index)  
2 Object remove(int index)
```

- Testarea:

```
1 boolean isEmpty()  
2 Boolean contains(Object elem)
```

- Aflarea poziției (eșec = -1):

```
1 int indexOf(Object e)  
2 int lastIndexOf(Object e)
```

Clasa ArrayList: Exemple

- Adăugarea unui element la orice poziție: $O(n)$.
- Accesarea unui element: $O(1)$.

```
1 import java.util.ArrayList;
2 import java.util.Collections;
3
4 public class ColectiiDemo {
5     public static void main(String[] args) {
6         ArrayList<Integer> a = new ArrayList<>();
7         a.add(10);
8         a.add(21);
9         a.add(-9);
10        a.add(10);
11        System.out.println("ArrayList contine: " + a);
12
13        Collections.sort(a);
14        System.out.println("ArrayList dupa sortare contine: " + a);
15    }
16 }
```

Diferențe între ArrayList<T> și Vector<T>

- ArrayList<T> și Vector<T> sunt similare în multe privințe.
- Diferențe:
 - Vector<T> este mai vechi și sincronizat.
 - ArrayList<T> face parte din cadrul general colecție și este mai eficient pentru uz general.

Clasa LinkedList

- Fiecare element este stocat separat în memorie.
- Caracteristici:
 - Inserările și ștergerile sunt eficiente: nu este nevoie de glisarea elementelor la inserare
 - Iterarea este mai lentă decât în `ArrayList`.
 - Nu se poate căuta aleator, trebuie traversată element cu element
- Exemple:

```
1 LinkedList<Integer> b = new LinkedList<>();  
2 b.add(10); //adauga la sfarsit  
3 b.add(21);  
4 b.addFirst(-9);  
5 b.addLast(-5);  
6 b.remove(2); //sterge elementul de pe pozitia 2  
7 System.out.println(b);
```

Clasa LinkedList: Exemplu 2

```
1 import java.util.LinkedList;
2
3 public class LinkedListDemo {
4     public static void main(String[] args) {
5         LinkedList<String> object = new LinkedList<>();
6         object.add("A");
7         object.add("B");
8         object.addLast("C");
9         object.addFirst("D");
10        object.add(2, "E");
11        object.add("F");
12        System.out.println("LinkedList: " + object);
13        // Linked list : [D, A, E, B, C, F, G]
14        object.remove("B");
15        object.removeFirst();
16        object.removeLast();
17        System.out.println("Lista dupa stergeri: " + object);
18        if (object.contains("E"))
19            System.out.println("Lista contine 'E'");
20        System.out.println("Dimensiune lista: " + object.size());
21    }
22 }
```

Clasa Stack: Exemplu

- Clasa Stack implementează o structură de date de tip stivă (**LIFO**).
- Se bazează pe clasa moștenită Vector, care nu este recomandată pentru aplicații moderne ce necesită sincronizare fină.
- Poate fi înlocuită cu alte implementări mai eficiente, cum ar fi Deque din Java Collections Framework.

```
1 import java.util.Stack;
2 public class StackDemo {
3     public static void main(String[] args) {
4         Stack<String> st = new Stack<>();
5         st.push("Afara");
6         st.push("ploua");
7         st.push("foarte");
8         st.push("tare");
9
10        String p = st.pop();
11        System.out.println("Stack contine: " + st);
12        //Afara ploua foarte
13    }
14 }
```

Interfața Queue: Exemplu

- Interfața Queue din Java este utilizată pentru a reprezenta o coadă (FIFO)

```
1 import java.util.LinkedList;
2 import java.util.Queue;
3
4 public class QueueDemo {
5     public static void main(String[] args) {
6         Queue<String> q = new LinkedList<>();
7         q.add("afara");
8         q.add("ploua");
9         q.add("foarte");
10        q.add("tare");
11
12        System.out.println("Coada: " + q);
13        String front = q.remove();
14        System.out.println("Element sters: " + front);
15        System.out.println("Coada dupa stergere: " + q);
16    }
17 }
```

Exemplu

Scrieți un program care citește de la tastatură un vector / listă de dreptunghiuri.

- Dacă laturile introduse nu formează un dreptunghi aruncați o excepție definită de utilizator (ExcepțieDreptunghi) și tratați excepția corespunzător.
- Calculați aria totală a tuturor dreptunghiurilor.
- Găsiți dreptunghiul cu aria maximă.
- Ordonăți dreptunghiurile după arie.
- Creați o nouă colecție care conține doar dreptunghiurile care sunt pătrate, ordonate crescător după arie.

Rezolvare - 1

```
1 import java.util.*;
2 class ExceptieDreptunghi extends Exception {
3     public ExceptieDreptunghi(String mesaj) { super(mesaj); }
4 }
5 class Dreptunghi implements Comparable<Dreptunghi> {
6     private double latime;
7     private double inaltime;
8     public Dreptunghi(double latime, double inaltime) throws
        ExceptieDreptunghi {
9         if (latime<=0 || inaltime<=0)
10             throw new ExceptieDreptunghi("Laturile tb sa fie pozitive.");
11         this.latime = latime;
12         this.inaltime = inaltime;
13 }
14 public double aria() { return latime * inaltime; }
15 public double perimetru() { return 2 * (latime + inaltime); }
16 public boolean ePatrat() { return Math.abs(latime - inaltime)<1e-9;}
17 @Override public int compareTo(Dreptunghi d) {
18     if (this.aria()< d.aria()) return -1;
19     if (this.aria()>d.aria()) return 1;
20     return 0; //ariile sunt egale
21 }
22 @Override public String toString() {
23     return String.format("L=% 2f H=% 2f | A=% 2f P=% 2f"
```

Rezolvare -2

```
1 public class Main {
2     public static void main(String[] args) {
3         Scanner scanner = new Scanner(System.in);
4         ArrayList<Dreptunghi> dreptunghiuri = new ArrayList<>();
5         System.out.print("Introduceti numarul de dreptunghiuri: ");
6         int n = scanner.nextInt();
7         // Citim n dreptunghiuri, tratand ExceptieDreptunghi
8         for (int i = 0; i < n; i++) {
9             System.out.println("Dreptunghiul " + (i + 1) + ":");
10            boolean dreptunghiValid = false;
11            while (!dreptunghiValid) {
12                System.out.print("    Latetime = ");
13                double latime = scanner.nextDouble();
14                System.out.print("    Inaltime = ");
15                double inaltime = scanner.nextDouble();
16                try {
17                    Dreptunghi d = new Dreptunghi(latime, inaltime);
18                    dreptunghiuri.add(d);
19                    dreptunghiValid = true; // iesim din while
20                } catch (ExceptieDreptunghi e) {
21                    System.out.println("Eroare: " + e.getMessage());
22                    System.out.println("Reintroduceti laturile pt acest dreptunghi");
23                    .");
24                }
25            }
26        }
27    }
28 }
```

Rezolvare

```
1 if (dreptunghiuri.isEmpty()) {
2     System.out.println("Nu s-au introdus dreptunghiuri valide.");
3     return;
4 }
5 // 1) Calculam aria totala
6 double ariaTotala = 0.0;
7 for (Dreptunghi d : dreptunghiuri) {
8     ariaTotala += d.aria();
9 }
10 // 2) Gasim dreptunghiul cu aria maxima
11 Dreptunghi max = dreptunghiuri.get(0);
12 for (Dreptunghi d : dreptunghiuri) {
13     if (d.aria() > max.aria()) max = d;
14 }
15 // 3) Ordonam dreptunghiurile dupa arie (crescator)
16 Collections.sort(dreptunghiuri); // foloseste Comparable
```

Rezolvare

```
1 // 4) Cream o noua colectie doar cu patrate, ordonate crescator dupa
   arie
2 ArrayList<Dreptunghi> patrate = new ArrayList<>();
3 for (Dreptunghi d : dreptunghiuri) { // nu folosim stream-uri
4     if (d.estePatrat()) patrate.add(d);
5 }
6 Collections.sort(patrate);
7 System.out.println("\nAria totala a tuturor dreptunghiurilor: " +
   ariaTotala);
8 System.out.println("Dreptunghiul cu aria maxima: " + max);
9 System.out.println("\nDreptunghiuri ordonate dupa arie (crescator):");
10 for (Dreptunghi d : dreptunghiuri) {
11     System.out.println(d);
12 }
13 System.out.println("\nDreptunghiuri care sunt patrate (ordonate
   crescator dupa arie):");
14 if (patrate.isEmpty()) {
15     System.out.println("Nu exista patrate.");
16 } else {
17     for (Dreptunghi p : patrate) System.out.println(p);
18 }
19 scanner.close();
20 }
```

Set

Colecții fără duplicate

- Se utilizează atunci când este nevoie de unicitatea elementelor stocate (nu sunt permise elemente duplicate)
- `HashSet<T>`, `LinkedHashSet<T>` sunt folosite pentru unicitate:
 - `HashSet`: Utilizează o tabelă hash pentru stocare, ceea ce face operațiile foarte rapide (complexitate aproape constantă: $O(1)$ pentru operațiile de bază: `add`, `remove`, `contains`).
 - `LinkedHashSet`: Păstrează ordinea de inserare (este mai lent decât `HashSet` din cauza gestionării ordinii, dar păstrează aceleași operații eficiente)
- Aplicații practice
 - `HashSet`: Eliminarea duplicatelor
 - `HashSet`: Verificarea rapidă a existenței unui element într-o colecție.
 - `LinkedHashSet`: stocarea elementelor unice într-o ordine predictibilă.
 - `LinkedHashSet`: gestionarea sesiunilor utilizatorilor unde ordinea de inserare este relevantă.

HashSet și LinkedHashSet: Exemplu

```
1 import java.util.HashSet;
2 import java.util.LinkedHashSet;
3
4 public class Main {
5     public static void main(String[] args) {
6         HashSet<String> hset = new HashSet<String>();
7         hset.add("afara");
8         hset.add("ninge");
9         hset.add("tare");
10        hset.add("azi");
11        System.out.println(hset); //[azi, tare, afara, ninge]
12        LinkedHashSet<String> lset = new LinkedHashSet<String>();
13        lset.add("afara");
14        lset.add("ninge");
15        lset.add("tare");
16        lset.add("azi");
17        System.out.println(lset); //[afara, ninge, tare, azi]
18    }
19 }
```

Cum funcționează HashSet și LinkedHashSet cu hashCode()?

- Organizare internă:
 - Atât HashSet, cât și LinkedHashSet se bazează pe HashMap pentru stocarea internă a elementelor.
 - Când un element este adăugat, codul său hash (hashCode()) este folosit pentru a determina bucket-ul în care este stocat elementul.
- Identificarea unicității:
 - Dacă două obiecte au același cod hash, HashSet și LinkedHashSet utilizează metoda equals() pentru a determina dacă obiectele sunt cu adevărat egale.
 - Dacă hashCode() și equals() confirmă că două obiecte sunt diferite, ele sunt stocate în locuri distincte, chiar dacă au același cod hash (coliziune).

Suprascrierea hashCode() în clase personalizate

Dacă stocăm obiecte personalizate în HashSet sau LinkedHashSet, trebuie să suprascriem hashCode() și equals() pentru a menține corectitudinea.

```
1 import java.util.HashSet;
2 class Person {
3     private String name;
4     private int age;
5     public Person(String name, int age) {
6         this.name = name;    this.age = age; }
7     @Override
8     public int hashCode() { return name.hashCode() * 31 + age; }
9     // Suprascriere equals
10    @Override
11    public boolean equals(Object obj) {
12        if (this == obj) return true;
13        if (obj == null || getClass() != obj.getClass()) return false;
14        Person person = (Person) obj;
15        return age == person.age && name.equals(person.name);
16    }
17    @Override
18    public String toString(){return "Person{name=' "+name+"', age="+age+"}
19        ";
20    } }
```

Suprascrierea hashCode() în clase personalizate

```
1 public class HashSetExample {
2     public static void main(String[] args) {
3         HashSet<Person> set = new HashSet<>();
4         // Adaugare obiecte
5         set.add(new Person("Alice", 30));
6         set.add(new Person("Bob", 25));
7         set.add(new Person("Alice", 30)); // Duplicat logic
8         // Afisare elemente
9         System.out.println("Elementele din HashSet: " + set);
10    }
11 }
12
13 /* Afisare:
14 Elementele din HashSet:
15 [Person{name='Bob', age=25}, Person{name='Alice', age=30}]
16 */
```

Rulati acelasi program, fara să suprascrieți metoda hashCode() si equals in clasa Person !

Colecții sortate

Interfața SortedSet și clasa TreeSet

- Caracteristici:
 - Pastreaza multimea sortata crescator (sortată în funcție de ordinea naturală a elementelor sale, sau după un Comparator)
- Clasa TreeSet implementează interfața SortedSet:
 - Utilizează o structură arbore binar de căutare (un arbore roșu-negru) pentru stocarea elementelor.
 - Elemente ordonate automat în ordine naturală sau conform unui comparator definit.
 - Operații precum adăugare, ștergere, căutare au complexitate $O(\log n)$

Utilizarea TreeSet cu ordonare naturală

```
1 import java.util.TreeSet;
2 public class TreeSetExample {
3     public static void main(String[] args) {
4         TreeSet<Integer> treeSet = new TreeSet<>();
5         // Adaugare elemente
6         treeSet.add(5);
7         treeSet.add(2);
8         treeSet.add(8);
9         treeSet.add(1);
10        // Afisare elemente [1, 2, 5, 8]
11        System.out.println("Elementele din TreeSet: " + treeSet);
12        // Afisare primul si ultimul element
13        System.out.println("Primul element: " + treeSet.first());
14        System.out.println("Ultimul element: " + treeSet.last());
15        // Submultimi
16        System.out.println("Elemente < 5: "+treeSet.headSet(5));
17        System.out.println("Elemente >= 5:"+treeSet.tailSet(5));
18    }
19 }
```

Utilizarea TreeSet cu comparator personalizat

```
1 import java.util.Comparator;
2 import java.util.TreeSet;
3 class Person {
4     private String name;
5     private int age;
6     public Person(String name, int age) {
7         this.name = name;
8         this.age = age;
9     }
10    public String getName() { return name; }
11    public int getAge() { return age; }
12    @Override
13    public String toString(){
14        return "Person{name='" + name + "', age=" + age + "}";
15    }
16 }
17 // Comparator personalizat pentru ordonare in functie de varsta
18 class AgeComparator implements Comparator<Person> {
19     @Override
20     public int compare(Person p1, Person p2) {
21         return Integer.compare(p1.getAge(), p2.getAge());
22     } }
```

Utilizarea TreeSet cu comparator personalizat

```
1 public class TreeSetWithCustomComparator {
2     public static void main(String[] args) {
3         // Crearea unui TreeSet cu comparatorul AgeComparator
4         TreeSet<Person> people = new TreeSet<>(new AgeComparator());
5
6         people.add(new Person("Alice", 30));
7         people.add(new Person("Bob", 25));
8         people.add(new Person("Charlie", 35));
9         people.add(new Person("Diana", 25)); // Aceeasi varsta ca Bob
10        // Afisarea elementelor
11        System.out.println("Persoanele ordonate dupa varsta:");
12        for (Person person : people) {
13            System.out.println(person);
14        }
15    }
16 }
17 /* se va afisa
18 Persoanele din TreeSet ordonate dupa varsta:
19 Person{name='Bob', age=25}
20 Person{name='Alice', age=30}
21 Person{name='Charlie', age=35}
22 */
```

Utilizarea TreeSet cu comparator personalizat

Dacă dorim să includem și numele în ordonare (când vârstele sunt egale), putem actualiza metoda `compare()` astfel:

```
1 @Override
2 public int compare(Person p1, Person p2) {
3     int ageComparison = Integer.compare(p1.getAge(), p2.getAge());
4     if (ageComparison != 0) {
5         return ageComparison;
6     }
7     return p1.getName().compareTo(p2.getName()); // Ordonare alfabetica
           dupa nume
8 }
```

Map

Cadrul general Map

- Map-urile gestionează perechi cheie-valoare.
- Caracteristici:
 - Cheile sunt unice.
 - Implementări frecvente: HashMap, TreeMap.
- Exemplele folosesc interfața Map<K, V>.

```
1 import java.util.HashMap;
2 import java.util.Map;
3 class Student {
4     private String name;
5     private int age;
6     // Constructor
7     public Student(String name, int age) { this.name = name; this.age =
        age; }
8     public String getName() { return name; }
9     public int getAge() { return age; }
10
11     public String toString() {
12         return "Student{name=' " + name + "', age=" + age + "}";
13     } }
```

Exemplu Map

```
1 public class HashMapExample {
2     public static void main(String[] args) {
3         // Crearea unui HashMap pentru a stoca obiecte Student
4         HashMap<Integer, Student> studentMap = new HashMap<>();
5         studentMap.put(101, new Student("Alice", 20));
6         studentMap.put(102, new Student("Bob", 22));
7         studentMap.put(103, new Student("Charlie", 21));
8         // Afisarea tuturor perechilor cheie-valoare
9         System.out.println("Perechi cheie-valoare in HashMap:");
10        for (Map.Entry<Integer, Student> entry : studentMap.entrySet())
11            System.out.print("Nr: "+entry.getKey()+" "+entry.getValue());
12        // Cautarea unui student dupa nr matricol
13        int searchKey = 102;
14        Student foundStudent = studentMap.get(searchKey);
15        if (foundStudent != null)
16            System.out.print("\nStudentul cu nr "+searchKey+": "+
17                               foundStudent);
18        else
19            System.out.print("\nStudentul cu nr "+searchKey+" nu exista.");
20        // Stergerea unui student din HashMap
21        studentMap.remove(103);
22        System.out.println("\nDupa stergerea studentului cu nr 103:");
23        System.out.println(studentMap);
24    }
25 }
```

Observații HashMap

- Dacă utilizăm obiecte personalizate ca chei, trebuie să suprascriem metodele `hashCode()` și `equals()` pentru a asigura comportamentul corect al `HashMap`.
- `HashMap` nu păstrează ordinea elementelor. Dacă ordinea este importantă, putem folosi `LinkedHashMap`.

Iteratori

Enumerări și iteratori

- Sunt obiecte folosite pentru a parcurge colecții.
- Tipuri:
 - Enumeration: de la JDK 1.0.
 - Iterator: introdus în JDK 1.2.
- Funcționează corect chiar dacă colecția se modifică.
- Obținerea unui enumerator:

```
1 Enumeration e = v.elements();  
2 while (e.hasMoreElements()) {  
3     System.out.println(e.nextElement());  
4 }
```

- Interfața `Iterator<T>`: Permite parcurgerea bidirecțională a unei liste.

```
1 interface Iterator<T> {  
2     public boolean hasNext();  
3     public T next();  
4     public void remove(); // optional  
5 }
```

- Iteratorul impune o ordine asupra elementelor colecției.
- Metode suplimentare: `add()`, `set()`, `previous()`

Folosirea unui iterator cu HashSet

```
1 import java.util.HashSet;
2 import java.util.Iterator;
3
4 public class Tst {
5     public static void main(String[] args) {
6         HashSet<String> s = new HashSet<>();
7         s.add("jingle");
8         s.add("bells");
9
10        Iterator<String> i = s.iterator();
11        while (i.hasNext())
12            System.out.println(i.next());
13    }
14 }
```

Exemplu ListIterator

```
1 import java.util.ArrayList;
2 import java.util.ListIterator;
3
4 public class ExempluListIterator {
5     public static void main(String[] args) {
6         ArrayList<Integer> al = new ArrayList<>();
7         for (int i = 0; i < 10; i++) {
8             al.add(i);
9         }
10
11        ListIterator<Integer> ltr = al.listIterator();
12        while (ltr.hasNext()) {
13            int i = ltr.next();
14            if (i % 2 == 0) {
15                ltr.set(i + 1);
16            }
17        }
18        System.out.println(al);
19    }
20 }
```

Capcană: Referințe întoarse de iteratori

- Iteratorii claselor colecție standard returnează referințe, nu copii.
- Modificarea unei valori returnate afectează colecția.
- Atenție la utilizarea obiectelor mutable.

Generice

Folosirea genericelor

- Un tip generic se definește în termenii unui alt tip pe care îl colectează sau asupra căruia acționează în vreun fel, folosind parantezele unghiulare (<>)
- Tipurile generice permit definirea colecțiilor care acceptă doar un anumit tip.

```
1 class Test<T> {
2     T obj;
3     Test(T obj) { this.obj = obj; }
4     public T getObject() { return this.obj; }
5 }
6 public class DemoGenerics {
7     public static void main(String[] args) {
8         Test<Integer> iObj = new Test<>(15);
9         System.out.println(iObj.getObject());
10        Test<String> sObj = new Test<>("JingleAllTheWay");
11        System.out.println(sObj.getObject());
12    }
13 }
14 /* Output: 15   JingleAllTheWay
15 */
```

Caractere de nume nespecificat (wildcards)

- Clasele și interfețele din cadrul general Collection pot avea specificări de parametri de tip care nu specifică complet tipul care îl va avea parametrul
- Pentru că ele specifică o gamă largă de tipuri de argumente, ele sunt numite caractere de nume nespecificat (wildcards)
- Wildcards permit specificarea incompletă a tipului parametrilor.
- Exemple:

```
1 void method(String arg1, ArrayList<?> arg2) { }
```

- În exemplul de mai sus, primul argument este de tipul String, în timp ce al doilea argument poate fi un *ArrayList* $< T >$ cu orice tip de bază
- Se poate folosi `<? extends T>` sau `<? super T>`.

Exemplu Wildcards

```
1 import java.util.Arrays;
2 import java.util.List;
3
4 public class ExempluWildcard {
5     public static void main(String[] args) {
6         List<Integer> list1 = Arrays.asList(1, 2, 3);
7         List<Double> list2 = Arrays.asList(1.1, 2.2, 3.3);
8         printList(list1);
9         printList(list2);
10    }
11
12    private static void printList(List<?> list) {
13        System.out.println(list);
14    }
15 }
```

Limitarea efectului unui wildcard

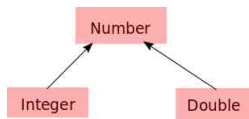
- Se poate limita efectul unui caracter de nume nespecificat (wildcard) precizând că tipul trebuie să fie un tip strămoș sau descendent al unei clase sau a unei interfețe
- Notăția `<?extendsString >` specifică faptul că argumentul care va fi folosit trebuie să fie un obiect din orice clasă descendentă a lui `String`
- Notăția `<?superString >` specifică faptul că argumentul care va fi folosit trebuie să fie un obiect din orice clasă parinte al lui `String`

Exemplu Wildcards: Extinderea unui tip

```
1 import java.util.Arrays;
2 import java.util.List;
3
4 public class ExempluWildcard2 {
5     public static void main(String[] args) {
6         List<Integer> list1 = Arrays.asList(14, 2, 3, 17);
7         System.out.println("Suma este: " + sum(list1));
8         //Suma este:36.0
9
10        List<Double> list2 = Arrays.asList(14.1, 52.1, 16.1);
11        System.out.println("Suma este: " + sum(list2));
12        // Suma este: este:82.300000000000001
13    }
14
15    private static double sum(List<? extends Number> list) {
16        double sum = 0.0;
17        for (Number i : list) {
18            sum += i.doubleValue();
19        }
20        return sum;
21    }
22 }
```

Exemplu Wildcards: Superclass

```
1 import java.util.Arrays;
2 import java.util.List;
3
4 public class ExempluWildcard3 {
5     public static void main(String[] args) {
6         List<Integer> list1 = Arrays.asList(4, 5, 6, 7);
7         printOnlyIntegerClassorSuperClass(list1);
8
9         List<Number> list2 = Arrays.asList(4.2, 5.3, 6.64, 7.3);
10        printOnlyIntegerClassorSuperClass(list2);
11    }
12
13    public static void printOnlyIntegerClassorSuperClass(
14        List<? super Integer> list) {
15        System.out.println(list);
16    }
17 }
```



Clasa Complex si Rational care implementeaza interfata Numeric

cu generice

```
1 public interface Numeric<T> {
2     T add(T other);
3 }
4 public class Complex implements Numeric<Complex> {
5     private double re;
6     private double im;
7
8     public Complex(double re, double im) {
9         this.re = re;
10        this.im = im;
11    }
12
13    @Override
14    public Complex add(Complex other) {
15        Complex rez = new Complex(this.re + other.re, this.im + other.im )
16        ;
17    }
18    public static void main(String[] args) {
19        Complex z1 = new Complex(1, 2);
20        Complex z2 = new Complex(3, -4);
21        Complex suma = z1.add(z2);
```